⚡ ElectraAI

Technical Architecture & Setup Documentation

AI-Powered Smart Search Engine

Version 1.0 | March 2026 | PKS Research & Development

Table of Contents

1. Project Overview

2. System Architecture

3. Technology Stack

4. Project Structure

5. Data Flow

6. Component Details

7. Local Setup Guide

8. Running Locally

9. Cloud Deployment

10. API Keys & Configuration

11. Troubleshooting

12. Next Steps

1. Project Overview

ElectraAI is an AI-powered smart search engine and chatbot built for B2B use cases, specifically designed for the electrical products industry. It uses Retrieval-Augmented Generation (RAG) to answer questions based on your own data — website content or product PDF catalogues.

What is RAG?

Retrieval-Augmented Generation (RAG) is an AI technique where instead of relying solely on the LLM's training data, the system first retrieves relevant content from your own database, then uses the LLM to generate accurate answers based on that content. This means ElectraAI answers questions using YOUR data, not generic internet knowledge.

Key Capabilities

- Scrapes and indexes website content automatically
- Ingests product PDF catalogues and datasheets
- Answers natural language questions about your products
- Runs locally on your Mac for development
- Deploys to the cloud for live production use
- Supports both local LLM (Ollama) and cloud LLM (Groq)

Use Cases

Use Case	Example Query	Data Source
Product Search	What MCBs are available under 10A?	PDF Catalogues
Pricing Queries	What is the price of switchgear X?	Price Lists
Technical Specs	What is the voltage rating of cable Y?	Datasheets
Availability	Is product Z in stock?	Inventory PDFs
Certification	Does product A have IEC certification?	Product Docs

2. System Architecture

ElectraAI is built on a two-mode architecture — Local Development Mode and Cloud Production Mode. Both modes share the same codebase and automatically detect the environment.

Architecture Diagram

```
USER QUERY
    |
    v
STREAMLIT UI  (chatbot/app.py)
    |
    v
HUGGINGFACE API  (converts query to vector)
    |
    v
CHROMADB  (searches for relevant chunks)
    |
    v
GROQ / OLLAMA LLM  (generates answer)
    |
    v
RESPONSE DISPLAYED TO USER
```

Two Operating Modes

Component	Local Mode (Dev)	Cloud Mode (Prod)
Chat UI	http://localhost:8501	Streamlit Cloud URL
LLM	Ollama + Mistral 7B	Groq API (llama-3.3-70b)

Component	Local Mode (Dev)	Cloud Mode (Prod)
Embeddings	HuggingFace Router API	HuggingFace Router API
Vector DB	ChromaDB (local files)	ChromaDB (GitHub repo)
Data Source	Local JSON / PDFs	GitHub repository

Data Ingestion Pipeline

```
WEBSITE / PDF FILES
    |
    v
SCRAPER / PDF LOADER  (engine/scraper.py or engine/pdf_loader.py)
    |  Extracts clean text from each page
    v
RAW DATA  (data/raw/website_data.json)
    |
    v
TEXT SPLITTER  (RecursiveCharacterTextSplitter)
    |  Splits into 500-word chunks with 50-word overlap
    v
HUGGINGFACE EMBEDDINGS  (all-MiniLM-L6-v2)
    |  Converts each chunk to a 384-dimension vector
    v
CHROMADB  (database/chroma_db/)
    |  Stores all vectors persistently
    v
READY FOR QUERIES ✅
```

3. Technology Stack

Core Components

Component	Technology	Purpose	Cost
LLM (Local)	Mistral 7B via Ollama	Generate answers locally	Free
LLM (Cloud)	Llama 3.3 70B via Groq	Generate answers on cloud	Free tier
Embeddings	all-MiniLM-L6-v2 via HuggingFace	Convert text to vectors	Free tier
Vector DB	ChromaDB	Store & search embeddings	Free
Framework	LangChain	Connect all AI components	Free
UI	Streamlit	Chat interface	Free
Hosting	Streamlit Cloud	Live deployment	Free
Code	GitHub	Version control & deployment	Free
Web Scraper	BeautifulSoup + Requests	Extract website content	Free
PDF Reader	PyPDF	Extract PDF content	Free

Python Libraries

Library	Version	Purpose
langchain	Latest	Core AI orchestration framework
langchain-community	Latest	Community integrations
langchain-groq	Latest	Groq LLM integration
langchain-ollama	Latest	Ollama LLM integration
langchain-huggingface	Latest	HuggingFace embeddings
langchain-text-splitters	Latest	Text chunking utilities
chromadb	Latest	Vector database
streamlit	Latest	Web UI framework
beautifulsoup4	Latest	HTML parsing for scraper
pypdf	Latest	PDF text extraction
python-dotenv	Latest	Environment variable management

Library	Version	Purpose
groq	Latest	Groq API client
huggingface-hub	Latest	HuggingFace API client

4. Project Structure

ElectraAI follows a clean modular structure separating data ingestion, AI pipeline, and UI components.

```
ElectraAI/                        # Root project directory
│
├── chatbot/                      # UI Layer
│   └── app.py                    # Streamlit chat interface
│
├── engine/                       # AI Pipeline Layer
│   ├── scraper.py                # Website crawler
│   ├── pdf_loader.py             # PDF ingestion (for product catalogues)
│   ├── rag_pipeline.py           # Chunking + embeddings + ChromaDB
│   └── test_retrieval.py         # Testing retrieval quality
│
├── data/                         # Data Layer
│   └── raw/
│       └── website_data.json     # Scraped website content
│
├── database/                     # Vector Database
│   └── chroma_db/                # ChromaDB persistent storage
│       ├── chroma.sqlite3
│       └── [vector files]
│
├── models/                       # Model configurations
├── logs/                         # Application logs
│
├── .env                          # API keys (never pushed to GitHub)
├── .gitignore                    # Files to exclude from Git
├── requirements.txt              # Python dependencies
├── runtime.txt                   # Python version for Streamlit Cloud
└── README.md                     # Project readme
```

File Responsibilities

File	Responsibility	When to Run
scraper.py	Crawls website, extracts clean text, saves to JSON	When website content changes
pdf_loader.py	Reads PDFs, extracts text, saves to JSON	When new PDFs are added
rag_pipeline.py	Chunks text, creates embeddings, stores in ChromaDB	After scraper/pdf_loader
app.py	Streamlit UI, handles user queries, returns answers	Always (main app)
test_retrieval.py	Tests if ChromaDB returns relevant results	When debugging quality

5. Data Flow

Query Processing Flow

When a user asks a question, ElectraAI processes it through these steps:

1. User types a question in the Streamlit chat interface
2. The question is sent to HuggingFace API which converts it to a 384-dimension vector
3. ChromaDB searches for the 3 most similar vectors in its database
4. The 3 matching text chunks are retrieved as context
5. The context + original question is formatted into a prompt
6. The prompt is sent to Groq API (or local Ollama)
7. Llama 3.3 70B generates a precise answer based on the context
8. The answer is displayed in the chat interface

Embedding Similarity Search

How Similarity Search Works

Every text chunk in your database has been converted to a vector (list of 384 numbers). When a user asks a question, that question is also converted to a vector. ChromaDB finds the chunks whose vectors are mathematically closest to the question vector. Closer vectors = more similar meaning. This is why ElectraAI can find relevant content even when the exact words don't match — it understands meaning, not just keywords.

Chunking Strategy

Parameter	Value	Reason
Chunk Size	500 characters	Small enough for precise retrieval, large enough for context
Chunk Overlap	50 characters	Prevents losing context at chunk boundaries
Top K Results	3 chunks	Provides enough context without overwhelming the LLM
Embedding Model	all-MiniLM-L6-v2	Fast, accurate, free, works via API

6. Component Details

Scraper (engine/scraper.py)

The scraper crawls your website and extracts clean, structured text. It removes navigation menus, headers, and footers to keep only meaningful product content.

Feature	Detail
Target URL	Configurable via BASE_URL variable
Max Pages	Configurable via MAX_PAGES (default: 50)
Content Extraction	Removes nav, header, footer noise
Encoding Fix	Fixes UTF-8 encoding issues automatically
Output	data/raw/website_data.json

RAG Pipeline (engine/rag_pipeline.py)

The RAG pipeline is the heart of ElectraAI. It transforms raw text into a searchable vector database.

Step	Component	Output
1. Load	JSON file reader	List of page documents
2. Chunk	RecursiveCharacterTextSplitter	121 text chunks
3. Embed	HuggingFace all-MiniLM-L6-v2	384-dim vectors per chunk
4. Store	ChromaDB	Persistent vector database

Chatbot UI (chatbot/app.py)

The Streamlit app provides the chat interface and orchestrates all components. It auto-detects whether it is running locally or on Streamlit Cloud.

Feature	Detail
Environment Detection	Checks /mount/src path to detect Streamlit Cloud
LLM Selection	Uses Groq if GROQ_API_KEY found, else Ollama
Caching	@st.cache_resource prevents reloading on each query
Chat History	Stored in st.session_state for conversation memory
Context Window	Top 3 relevant chunks sent to LLM

7. Local Setup Guide

Follow these steps to set up ElectraAI on a fresh Mac. This guide assumes macOS Big Sur or later.

Prerequisites

Tool	Version	Purpose	Install Command
macOS	11+ (Big Sur or later)	Operating system	System upgrade
Homebrew	4.0+	Mac package manager	/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
Python	3.11 - 3.13	Programming language	brew install python@3.11
VS Code	Any	Code editor	brew install --cask visual-studio-code
Git	Any	Version control	brew install git
Ollama	0.18+	Local LLM runner	brew install ollama

Step 1 — Install Xcode CLI Tools

Required for compiling Python packages with C extensions.

```
xcode-select --install
```

A popup will appear. Click Install and wait (~5 minutes).

Step 2 — Install Homebrew

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Step 3 — Install Python

```
brew install python@3.11
python3 --version  # Verify: should show Python 3.11.x
```

Step 4 — Install VS Code

```
brew install --cask visual-studio-code
# Then open VS Code and run: Shell Command: Install 'code' command in PATH
```

Step 5 — Install Ollama

```
brew install ollama
ollama --version  # Verify installation
```

Step 6 — Pull AI Models

Pull the LLM and embedding model. Mistral is ~4GB, nomic-embed-text is ~500MB.

```
# Start Ollama service (keep this terminal tab open)
ollama serve

# In a new terminal tab, pull models:
ollama pull mistral
ollama pull nomic-embed-text

# Test Mistral is working:
ollama run mistral
# Type a message, press Enter, press Ctrl+D to exit
```

Step 7 — Clone Repository

```
git clone https://github.com/PKS-Research-Development/ElectraAI.git
cd ~/Desktop/ElectraAI
```

Step 8 — Create Virtual Environment

A virtual environment isolates project dependencies from your system Python.

```
python3 -m venv venv
source venv/bin/activate
# You should see (venv) in your terminal prompt
```

Step 9 — Install Dependencies

```
pip install --upgrade pip
pip install langchain langchain-community langchain-ollama langchain-groq
pip install langchain-core langchain-text-splitters langchain-huggingface
pip install chromadb beautifulsoup4 requests streamlit
pip install pypdf python-dotenv groq huggingface-hub transformers

# Verify installation:
python3 -c "import langchain; import chromadb; import streamlit; print('All good!')"
```

Step 10 — Configure API Keys

Create a .env file in the project root with your API keys:

```
# Create .env file

touch ~/Desktop/ElectraAI/.env


# Add these keys to .env:

GROQ_API_KEY=gsk_xxxxxxxxxxxxxxxxxxxxxxxxxxxx

HF_TOKEN=hf_xxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

How to Get API Keys

GROQ_API_KEY: Sign up at https://console.groq.com → API Keys → Create API Key → Name it ElectraAI → Copy key.

HF_TOKEN: Sign up at https://huggingface.co → Settings → Access Tokens → New Token → Role: Read → Copy key.

8. Running ElectraAI Locally

First Time Setup

Follow these steps in order the very first time you run ElectraAI:

Terminal Tab 1 — Start Ollama Service

```
ollama serve

# Keep this tab open throughout your session
```

Terminal Tab 2 — Run the Data Pipeline

```
cd ~/Desktop/ElectraAI

source venv/bin/activate


# Step 1: Scrape website data

python3 engine/scraper.py


# Step 2: Build vector database

python3 engine/rag_pipeline.py


# Step 3: Launch chatbot

streamlit run chatbot/app.py
```

Open your browser and go to: http://localhost:8501

Daily Development Workflow

After first-time setup, your daily workflow is much simpler:

9. Open Terminal
10. Tab 1: Run ollama serve
11. Tab 2: cd ~/Desktop/ElectraAI && source venv/bin/activate
12. Tab 2: streamlit run chatbot/app.py
13. Open http://localhost:8501 in browser

Using PDF Data Instead of Website

To use your own product PDFs instead of a scraped website:

```
# Step 1: Place all PDF files in:
# ~/Desktop/ElectraAI/data/pdfs/

# Step 2: Run PDF loader instead of scraper
python3 engine/pdf_loader.py

# Step 3: Rebuild vector database
rm -rf ~/Desktop/ElectraAI/database/chroma_db
python3 engine/rag_pipeline.py

# Step 4: Restart chatbot
streamlit run chatbot/app.py
```

Testing Retrieval Quality

If answers seem irrelevant, test what ChromaDB is returning:

```
python3 engine/test_retrieval.py
# This shows the exact chunks being retrieved for a test query
# Helps diagnose poor answer quality
```

Environment Variables Reference

Variable	Required	Description	Where to Get
GROQ_API_KEY	Yes (cloud LLM)	Groq API key for Llama 3.3 70B	console.groq.com
HF_TOKEN	Yes (embeddings)	HuggingFace API token	huggingface.co/settings/tokens

9. Cloud Deployment

ElectraAI is deployed on Streamlit Cloud, connected to GitHub for automatic deployments.

Deployment Architecture

```
Developer Mac
     |
     | git push
     v
GitHub Repository (PKS-Research-Development/ElectraAI)
     |
     | Auto-deploy webhook
     v
Streamlit Cloud
     |  Installs requirements.txt
     |  Reads secrets (API keys)
     |  Runs chatbot/app.py
     v
Live URL: electraai.streamlit.app
```

Deployment Steps

14. Push code to GitHub: git add . && git commit -m 'message' && git push
15. Go to https://share.streamlit.io
16. Click New App → Select repository PKS-Research-Development/ElectraAI
17. Set Main file path to chatbot/app.py
18. Click Advanced Settings → Add secrets (API keys)
19. Click Deploy

Streamlit Cloud Secrets

Add these in Streamlit Cloud → App Settings → Secrets:

```
GROQ_API_KEY = "gsk_xxxxxxxxxxxxxxxxxxxxxxxxxxxx"

HF_TOKEN = "hf_xxxxxxxxxxxxxxxxxxxxxxxxxxxx"
```

Redeploying After Changes

```
# Push changes to GitHub

git add .

git commit -m "Description of changes"

git push


# Streamlit auto-redeploys on push

# Or manually: Streamlit Cloud → App → Menu → Reboot app
```

10. API Keys & Configuration

Required API Keys

Key	Service	Free Tier	Get It At
GROQ_API_KEY	Groq (LLM)	Yes - generous free tier	console.groq.com
HF_TOKEN	HuggingFace (Embeddings)	Yes - free tier available	huggingface.co

Model Configuration

Setting	Current Value	Where to Change	Notes
LLM Model	llama-3.3-70b-versatile	chatbot/app.py	Groq's best free model
Embedding Model	all-MiniLM-L6-v2	rag_pipeline.py + app.py	Fast, accurate, free
Chunk Size	500	rag_pipeline.py	Rebuild DB after changing
Top K Results	3	chatbot/app.py	More = better context, slower
Max Scrape Pages	50	scraper.py	Increase for larger sites

11. Troubleshooting

Error	Cause	Fix
ConnectionError: Ollama	Ollama service not running	Run: ollama serve in a terminal tab
ModuleNotFoundError	Missing Python package	Run: pip install <package-name>
403 GitHub push error	Wrong credentials cached	Clear Keychain Access GitHub entries
ChromaDB KeyError: 0	Deprecated embedding class	Use HuggingFaceEndpointEmbeddings
410 Gone HuggingFace	Old API URL	Use router.huggingface.co/hf-inference/
Streamlit onnxruntime error	Version pinned in requirements.txt	Use unpinned requirements.txt
Poor answer quality	Noisy scraped data or wrong chunks	Run test_retrieval.py to debug
Slow responses locally	RAM pressure from other apps	Close Chrome tabs, run: sudo purge
App not loading on cloud	Missing secret keys	Add keys in Streamlit Cloud secrets

12. Next Steps & Roadmap

Immediate Improvements

- Replace books.toscrape.com test data with real electrical product PDFs
- Add company logo and branding to Streamlit UI
- Increase MAX_PAGES in scraper for complete website coverage
- Add password protection for internal use

Feature Enhancements

- Add conversation memory so the chatbot remembers previous questions
- Add source citations showing which page/PDF each answer came from
- Add a feedback button (thumbs up/down) to improve answer quality
- Add analytics dashboard to track popular questions
- Support multiple languages for international customers

Technical Improvements

- Set up automatic re-scraping when website content changes
- Add PDF auto-ingestion pipeline for new product catalogues
- Implement hybrid search (vector + keyword) for better accuracy
- Add caching for frequent queries to improve response speed

Production Readiness

- Add user authentication and role-based access
- Set up monitoring and error alerting
- Implement rate limiting to prevent API abuse
- Add backup and recovery for ChromaDB database

⚡ **ElectraAI Documentation v1.0**

Built by PKS Research & Development | March 2026

Powered by Groq + HuggingFace + ChromaDB + LangChain + Streamlit